

SSA Register Allocation for M6809

<https://gemini.google.com/app/dd41a1e414ce372c>

User prompt: We have a compiler written in Go for a low-level C-like language. The compiler uses SSA in the Intermediate Representation (IR) code. The M6809 back end we have written is very inefficient. I want to do smarter register allocation. What is the modern approach to translating IR blocks to Assembly Language with optimal register allocation?

Response: Translating SSA (Static Single Assignment) IR to an 8-bit architecture like the M6809 is notoriously difficult because "modern" register allocation algorithms (designed for x86 or ARM) assume a large, uniform register file. The M6809, by contrast, has a highly **non-orthogonal** architecture with specialized registers (A, B, D, X, Y, U, S).

The modern consensus for SSA-based compilers is a **Decoupled Register Allocation** approach. Instead of one massive pass, you break the problem into three distinct phases.

1. The SSA-Based Modern Approach

Since your IR is already in SSA form, you can take advantage of the fact that the **interference graphs of SSA programs are chordal graphs**. This allows for "optimal" coloring in linear time, which is much more efficient than the traditional NP-complete graph coloring used in the 90s.

Phase A: Spilling (Lowering Register Pressure)

Before assigning registers, you must ensure the "register pressure" (the number of variables live at any one point) is less than or equal to the number of available physical registers.

- **The Approach:** Use a simple **Linear Scan** to identify variables to "spill" to the stack.
- **M6809 Nuance:** Since the 6809 has so few registers, you should aggressively spill variables that are only used far in the future.

Phase B: SSA-Based Coloring

Once the pressure is low enough, you assign "virtual registers" to physical ones.

- **Algorithm:** Use **Greedy Coloring** with a **Maximum Cardinality Search (MCS)** ordering. Because SSA interference graphs are chordal, this greedy approach will find the minimum number of registers needed without backtracking.

Phase C: SSA Reconstruction & Coalescing

SSA form naturally creates many "copy" instructions (especially during ϕ -node elimination).

- **Coalescing:** Use **Chaitin-Briggs Conservative Coalescing** to merge virtual registers that are connected by a move/copy instruction, provided they don't interfere. This eliminates redundant TFR or EXG instructions.

2. Specialized Strategies for M6809

A generic allocator will still produce slow code on a 6809 because it doesn't understand that X and Y are for pointers, while A and B are for math. You need **Target-Specific Constraints**.

Register Class Partitioning

Don't treat all registers as equal. Divide them into "classes" in your back end:

- **Accumulators (A, B, D):** Best for 8-bit and 16-bit arithmetic.
- **Index Registers (X, Y):** Best for pointer math and LD / ST operations.
- **User Stack (U):** Often used as a third index register or a frame pointer.

Register Preferences (Biasing)

During the coloring phase, implement **Register Preferencing**. If a virtual register is used as a pointer (e.g., in a Load instruction), "bias" the allocator to choose X or Y . If it's used for an 8-bit add, bias it toward A or B .

Pattern Matching with "D"

The 6809 can treat A and B as a single 16-bit register D . Your IR-to-Assembly translator should look for 16-bit operations in the SSA and map them directly to D to avoid manual 8-bit carry propagation.

3. Comparison of Allocation Strategies

Algorithm	Complexity	Code Quality	Best Use Case
Linear Scan	$O(n)$	Good	Fast compilation; great for JITs or simple 8-bit scripts.
Graph Coloring	$O(n^2)$	Excellent	Standard for "Ahead-of-Time" (AOT) production compilers.
SSA-Chordal	$O(n + e)$	Optimal	The Modern Standard. Uses SSA properties to get Graph Coloring quality at Linear speeds.

Implementation Tip for Go

Since you are using Go, look at the `internal/ssa` package in the Go source code. Go's own compiler uses a custom register allocator that handles "register classes" and "clobbering" (where an instruction destroys the value in a register).

Critical Step: Before allocating, perform ϕ -node elimination (CSSA - Conventional SSA). If you don't do this correctly, you'll end up with "Lost Updates" where the wrong value is moved into a register at a block boundary.

User prompt: How do I model overlapping registers like the M6809 D register (A and B) in a register interference graph?

Response: Modeling overlapping registers—like the 6809's 16-bit D register which physically consists of the 8-bit A (high byte) and B (low byte) accumulators—is a classic challenge in register allocation. If the allocator assigns a virtual register to D , it must automatically prevent other variables from using A or B at the same time.

The modern standard to handle this is through **Register Aliasing** using **Matrix-based Interference** or **Weights/Quantas** within your Register Interference Graph (RIG).

Here is how you can implement this cleanly in a Go-based backend.

1. The Multi-Node Representation (Explicit Interferences)

The most straightforward way to model this in your graph is to create distinct nodes for every physical register, but explicitly draw **permanent interference edges** between overlapping registers.

- **Pre-colored Nodes:** Your RIG should start with a set of "pre-colored" nodes representing the physical hardware: `node_A`, `node_B`, and `node_D`.
- **Alias Edges:** Before you even look at your SSA IR, you hardcode interference edges between them:
 - `node_D` connects to `node_A`
 - `node_D` connects to `node_B`

How it works during coloring:

If a 16-bit SSA variable V_{16} is colored with D , that color becomes unavailable to any interfering node. Because D is pre-linked to A and B , any 8-bit variables V_8 that interfere with V_{16} will see that A and B are also blocked, preventing a collision.

2. The Weight-Based (Chaitin) Approach

If you are using a traditional Chaitin-style graph coloring allocator, you calculate the **degree** of a node to determine if it can be safely colored. Overlapping registers break standard degree calculation because a 16-bit allocation "squeezes" the register file harder than an 8-bit allocation.

To fix this, assign an **interference weight** (ω) to the edges based on the register size (often measured in bytes):

- An edge between two 8-bit variables has a weight of **1**.
- An edge between an 8-bit variable and a 16-bit variable (D) has a weight of **1**.
- An edge between two 16-bit variables has a weight of **2**.

When simplifying the graph, a node is considered "safe to color" if the sum of the weights of its remaining edges is less than the total capacity of the target register class.

3. Modeling in Go: The Bitmask Architecture

Because you are writing your compiler in Go, the most performant and modern way to handle this is to abandon explicit graph edges for physical registers and use **bitmasks** to represent register availability.

Instead of tracking colors as single integers, define each physical register as a bitmask of its underlying architectural resource.

```
type RegMask uint8

const (
    RegA RegMask = 1 << 0 // 0b01 (Resource: High Byte)
    RegB RegMask = 1 << 1 // 0b10 (Resource: Low Byte)
    RegD RegMask = RegA | RegB // 0b11 (Resource: Both Bytes)
)
```

The Allocation Algorithm Step:

When your tracker allocates a physical register to an SSA live range, it performs a bitwise OR into a busy mask for that basic block's current point:

```
// Check if a register is free
if (currentLiveMask & RegD) == 0 {
    // Allocate D, which automatically blocks A and B
    currentLiveMask |= RegD
}
```

This completely eliminates the need to build complex alias edges in your graph. If D is active (0b11), an attempt to allocate A (0b01) or B (0b10) will fail the bitwise check (0b11 & 0b01) $\neq 0$.

4. Sub-Register Insertion/Extraction in SSA

To make this work seamlessly with your IR, your compiler's frontend/midend must explicitly tell the backend when it is breaking D apart or putting it together.

In your SSA IR, introduce two target-specific lowering operations:

1. `GetHighByte(V16) -> V8` (Maps later to zero-cost or simple TFR operations if already in *D*)
2. `CombineBytes(V8_High, V8_Low) -> V16`

During the SSA coloring phase, your coalescer should attempt to tie the virtual register of `V8_High` to *A* and `V8_Low` to *B*. If successful, the `CombineBytes` instruction completely vanishes from the emitted assembly, because the values are already sitting in the correct halves of *D*.
